System Tests and Others

In addition to ActiveSupport::TestCase, Rails provides some other subclasses of Minitest::TestCase that provide custom functionality. In most of these cases, the RSpec functionality you've already seen is built on top of these classes:

- ActionDispatch::SystemTestCase is the parent class for system tests, which you saw in Chapter 8, *Integration Testing with Capybara and Cucumber*, on page 151.
- ActionDispatch::IntegrationTest is wrapped by RSpec request specs, as seen in Chapter 11, *Testing Rails Display Elements*, on page 217.
- ActionView::TestCase, ActionMailer::TestCase, and ActiveJob::TestCase all provide specific support similar to the RSpec support discussed in Chapter 11, *Testing Rails Display Elements*, on page 217.

Let's look at the system and integration tests because they are the ones the Rails core uses most frequently.

Minitest System Tests

In Minitest, the Rails core team recommendation is to use system tests and Capybara only when invoking a JavaScript runtime; in other cases, the team recommends using integration tests.

You have one JavaScript test in your suite. Here's what it looks like in Minitest. You need to specify that you're using headless Chrome as your driver (the Rails default is Selenium with a visible browser):

minitest/02/test/application_system_test_case.rb

```
require "test_helper"

class ApplicationSystemTestCase < ActionDispatch::SystemTestCase
  driven_by :selenium_chrome_headless
end
```

The driven_by method takes the same options you listed in Chapter 8, *Integration Testing with Capybara and Cucumber*, on page 151.

Here's the test itself:

minitest/02/test/system/tasks_test.rb

```
require "application_system_test_case"

class TasksTest < ApplicationSystemTestCase
  test "reordering a task" do
    project = FactoryBot.create(:project, name: "Project Bluebook")
    FactoryBot.create(
      :task, project: project, title: "Search Sky", size: 1, project_order: 1)
```